SeparateChainingHashtable-master

CSCI 340 - Data Structures and Algorithm Analysis

Data Structure: Hash Table
Programming Focus: C++ development, construction of hash table using STL vector

Hashing with Separate Chaining and Indirect Sorting

For this computer assignment, you are to write a C++ program to create, search, print, and sort an item inventory. The item inventory information will be kept in a sequentially allocated table with a given size of entries with the default size TBL_SZ = 31. Each table entry is described by the following structure, as specified in the header file **entry.h**.

```
struct Entry{ string key; string desc; unsigned num; };
```

The `key` field is the item identifier, which is two uppercase letters followed by a decimal digit (e.g., AD5 or XR8). The `desc` field contains the item description, and the `num` field contains the number of copies of an item in the inventory. The item table can be accessed directly using the separate chaining technique to resolve the collisions. The private hash function expects an item key as the input argument, and it returns an integer in the range: **0 … (TBL_SZ−1)**, which has the following prototype: `int HT::hash(const string& key)` and is implemented in **hash.cc** you **may not** modify the hash function.

The item inventory system is defined as a class, named as **HT**, and its definition is given in the header file **htable.h**, which is in the same directory with **entry.h**. The hash table **hTable** (container of type `vector<list<Entry>>`) is used to store the entries of the item inventory, and the pointer table **pTable** (container of type `vector<Entry *>`) is used to store the addresses of the entries in **hTable**.

The member functions of HT are described below:

- `HT::HT(const unsigned &hs)` The hash table, which is a **vector of list** containers, can be created dynamically for a given fixed size `hs` by its constructor. Initially, the pointer table, which is a **vector of type Entry ***, is empty but its size will increase dynamically after inserting a new `Entry` item in this table, which is the address of the inserted item in the corresponding list container.
- `HT::~HT()` Since the hash table is implemented as a **vector of list containers**, the destructor of the hash table first frees the memory for all list containers and then frees the memory for the vector containers for the hash table and the pointer table.
- `void HT::insert(const Entry &e)` This public function inserts the record of the item e:(key, desc, num) in the hash table. If the key already exists in the table, then the function prints an error message; otherwise, it prints the index value of the inserted record in the hash

table and it also inserts the address of the record (in the hash table) into the pointer table. Since each element of the hash table is implemented as a list container with the data type `Entry`, in the case of a collision, simply insert the new record at the beginning of the corresponding list container. To check if the record `e` is already in the hash table, you can use the function `find_if()` in the STL. To compare the key of the record `e` with the keys of the elements in the list container, `list<Entry> &l = hTable[i]`, in hash table position `i`, you can use either a predicate or a lambda for the compare component of the `find_if()` function. If the item is a new item, then the `find_if()` function returns `l.cend()`.

- `void HT::search(const string& key)` This public function searches the hash table for a record with key. As in the `insert()` function, you can use the `find_if()` function from in the STL to search for a record in the hash table. If the search is successful, `search()` function prints the information for the record; otherwise, it prints an error message.
- `void HT::hTable_print()` This public function prints the subscript and the contents of all (and only) the active records in the hash table.
- `void HT::pTable_print()` This public function prints the contents of all (and only) the active records in the hash table. Since the records need to be printed in alphabetical order with their key values, this function first sort the elements of the pointer table using the `sort()` function in the STL. The `cmp` component for sorting, `bool cmp(Entry *p, Entry *q)`, returns **true** if the key of the item in location `p` comes before the item in location `q` in alphabetical order.

The input data consists of three types of records, as described below:

- Insertion records have the format `A:item-key:item-number:item-description:`
- Search records have the format `S:item-key:`
- Table print records have the format `P:`

The records stored in the inventory system have unique keys. Duplicate keys are not permitted.

The `main()` routine in the driver program **hashmain.cc** (provided), implements the following tasks:

1. Prints a header message.
2. Creates an empty hash table.
3. Reads the entire input file, and process each record as follows: if the record transaction code is:
    - **A** searches the hash table for a record having the same key. If such a record is found, prints an appropriate "*duplicate key*" message, and ignores the information from the input record. Otherwise, inserts the record in the hash table, and places the address of the record in the pointer table.
    - **S** searches the hash table for a record with a given key and prints the key and other information for the record. If the record is not found in the table, it prints an appropriate "*key not found*" message.
    - **P** prints the subscript and contents of each active record in the hash table.

4. After the data in the input file has been read, and the tables have been constructed, sorts the records in the pointer table, and then prints the records in the hash table in ascending order with the key field.
5. Prints a program termination message.

Assignment Notes:

- Include any necessary headers and add necessary global constants.
- You are not allowed to use any I/O functions from the C library, such as scanf or printf. Instead, use the I/O functions from the C++ library, such as cin or cout.
- Add documentation to the appropriate source files as discussed in your class.
- Prepare your Makefile (you need to construct and add Makefile) so that the TA only needs to invoke the command make to compile your source file and produce the executable file **hash**. This will then compile **hashmain.cc** (contains the `main()` routine and some auxiliary functions for reading and printing data) of the driver program and your source file **htable.cc** (contains your implementations of the member functions of HT). Make sure you use exactly the same file names specified here, i.e. **hash**, in your Makefile, otherwise your **submission will get 0 points**. You can test your program with sample datasets **hashsample01.in** and **hashsample02.in** and compare it against the two sample outputs **hashsample01.refout** and **hashsample02.refout**.

When your program is ready for grading, ***commit*** and ***push*** your local repository to remote git classroom repository and follow the ***Assignment Submission Instructions***.